

Performance engineering with your matmul benchmarks

Oct 3, 2026 · @Tomo

How to use this guide

Your benchmark suite asks four questions about your matrix multiply. This guide teaches the hardware and measurement ideas behind each answer, so you can read the charts, predict the next ones, and explain them in an interview.

Question	Benchmarks	Chart in bench/plots
How much does cache blocking help, and why?	BM_Naive , BM_Blocked , n = 100 to 4000	throughput_vs_size , cache_misses
Which tile size is best, and why?	BM_Blocked , BM_BlockedParallel , tile edge 8 to 512	tile_sweep
How well does the threadpool scale? Do 16 workers help on 8 cores?	BM_BlockedParallel , 1 to 16 workers at n = 2000	thread_scaling
How far is it from a tuned library?	BM_OpenBLAS at the same points	headline

The sections build on each other:

1. Sections 1 to 5 build a model of the machine: the two limits on any program, then the memory hierarchy, one core, SIMD, and many cores.
2. Sections 6 and 7 cover measurement: how the suite avoids fooling you, and how hardware counters work.
3. Sections 8 to 11 apply both: your results, the next optimizations, exercises, and interview prep.
4. Section 12 lists books, courses and papers for going deeper, by topic.

Conventions used throughout:

- Numbers come from the first full run (commit 22af068, five repetitions) on your Ryzen 7 9800X3D under WSL2. That run was noisy, so treat them as preliminary.
- "GPU side" paragraphs map each idea onto CUDA, starting from your tiled matmul kernel. They teach both directions: the GPU idea explains the CPU one, and the other way around.
- CS:APP means Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd edition. Section numbers like CS:APP 5.9 refer to it.
- An FMA is one multiply-add, `c += a * b`. Your $n \times n$ multiply does n^3 of them, which is $2n^3$ floating-point operations (FLOPs).

1. The two limits: compute and memory

Every program is limited either by how fast the chip does arithmetic or by how fast it moves data. Matrix multiply is one of the few programs that can hit the arithmetic limit. On your 9800X3D, one core peaks at about 160 GFLOP/s, and DRAM delivers about 56 GB/s.

The compute limit

Peak FLOP/s is cores $\times$ FLOP per cycle $\times$ clock rate. One Zen 5 core can start 2 FMA instructions per cycle, each on a 512-bit register of 8 doubles, and each FMA counts as 2 FLOPs. That is $2 \times 8 \times 2 = 32$ FLOP per cycle.

A microbenchmark on your machine confirmed it: 12 independent chains of 512-bit FMAs ran at exactly 0.50 cycles per FMA. At the 5.0 GHz the counters report, that is 160 GFLOP/s per core, and about 1.3 TFLOP/s for 8 cores if they held that clock. Your blocked kernel does 7.3 GFLOP/s, 4.5% of one core. OpenBLAS does 129, or 80%.

The memory limit

Reading a 2 GiB array, one core pulls about 56 GB/s from DRAM. Sixteen threads only reach 58 GB/s. Bandwidth is shared by all cores, and on this chip one core nearly saturates it. A kernel limited by bandwidth will not speed up with more threads.

Arithmetic intensity decides which limit you hit

Arithmetic intensity is FLOPs per byte moved to or from DRAM. A kernel can only reach peak if it does enough work per byte: peak $\div$ bandwidth, about $160 \div 56 \approx 2.9$ FLOP per byte for one core.

- **Vector add**, $c[i] = a[i] + b[i]$, does 1 FLOP per 24 bytes (two loads, one store), or 0.04 FLOP/byte. It is memory-bound on every computer ever built.
- **Matrix multiply** does $2n^3$ FLOPs on $3n^2$ doubles. If each matrix crossed DRAM once, that is $2n^3 \div 24n^2 = n/12$ FLOP per byte: 167 at $n = 2000$. Each element is reused n times. That reuse is why matmul can run at peak.

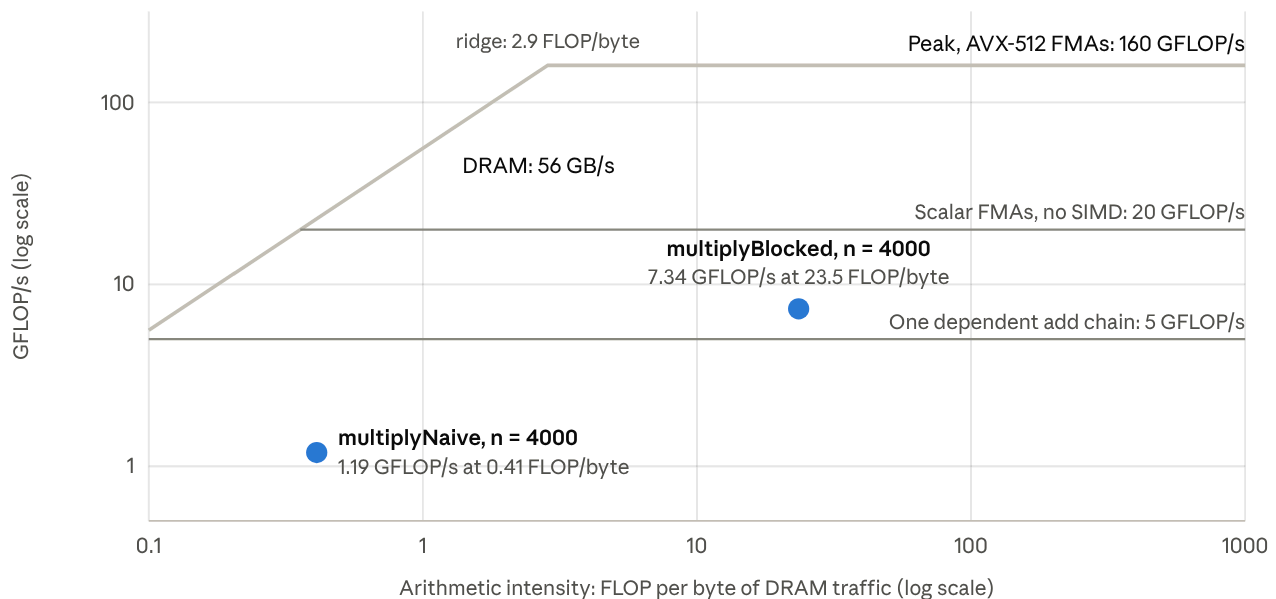
The catch is the "if". The naive loop does not reuse data while it is still in cache, so it moves far more than $24n^2$ bytes. Cache blocking is how you turn the theoretical reuse into actual reuse.

The roofline model

The roofline model, from [Williams, Waterman and Patterson](#) (Communications of the ACM, 2009), puts both limits on one log-log plot. Attainable GFLOP/s is the lower of peak and intensity $\times$ bandwidth. Lower horizontal lines, called ceilings, show the limit when you skip a hardware feature: SIMD, or independent work for the FMA units.

Both kernels sit far below this core's roofs

One Zen 5 core at about 5.0 GHz. Kernel intensity estimated from DRAM-fill counters.



measured on your 9800X3D: peak from an FMA throughput test, bandwidth from reading a 2 GiB array, kernels from the suite's first full run

Both kernels sit far below both roofs. Naive, in the memory-bound zone, gets 1/20 of what the bandwidth allows: it is not short of bandwidth, it waits on each miss. Blocked, past the ridge, gets 1/22 of peak and sits near the one-chain ceiling. Sections 3 and 4 explain those ceilings.

GPU side. Nsight Compute draws this same roofline for a CUDA kernel. Your shared-memory tiling did for global memory what cache blocking does for DRAM: each tile is loaded once and reused TILE times from on-chip memory, which raises arithmetic intensity by a factor of TILE. A GPU has a much higher roof and more bandwidth than a CPU, but the same question decides your speed: which line are you under?

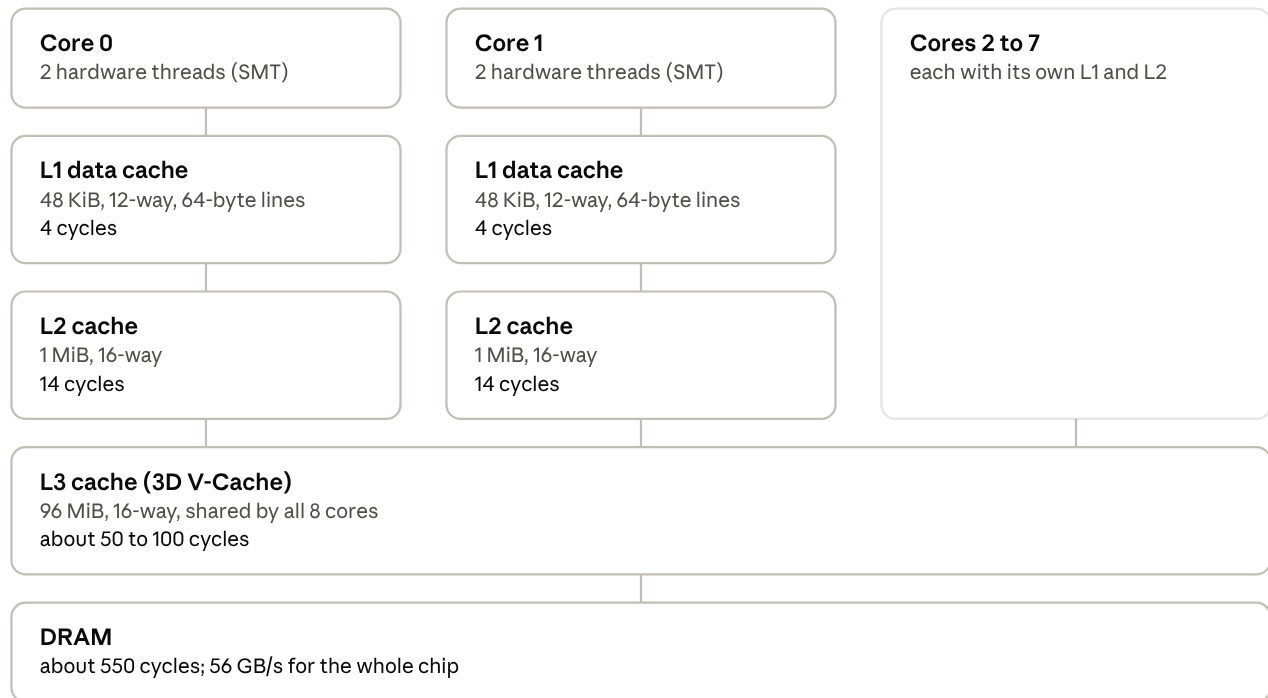
To go deeper: the roofline paper is short and readable. Hennessy and Patterson, *Computer Architecture: A Quantitative Approach* (6th edition), chapter 4 covers rooflines, SIMD and GPUs side by side.

2. The memory hierarchy

Data lives in a stack of caches, each bigger and slower than the one above it. On your machine a load costs 4 cycles from L1 and about 550 from DRAM, so a fast kernel is one that finds its data near the top.

A load from DRAM costs about 140 loads from L1

Ryzen 7 9800X3D. Latencies from a random pointer chase run on this machine.



cache hierarchy of the Ryzen 7 9800X3D · sizes from Linux sysfs, latencies measured on this machine

The latencies come from a random pointer chase: each load's address comes from the previous load, so nothing overlaps and the prefetchers cannot guess. Past a few MiB they also include TLB misses, covered below. Section 7 shows the full calibration.

Cache lines and locality

- Memory moves in 64-byte **lines**, 8 doubles each. Touch one double, and the other seven arrive with it.
- **Spatial locality** means using those neighbors soon. A walk along a row uses all 8. A walk down a column uses 1 per line it touches.
- **Temporal locality** means reusing data before it is evicted. Whether it is still cached depends on how much other data you touched in between.

In `multiplyNaive`, the `k` loop walks row `i` of A, which is contiguous, and column `j` of B, which touches a new line per `k`. Those B lines get reused for `j + 1` to `j + 7` only if all `n` of them survive in L1. At `n = 500` they fit (32 KiB); at `n = 1000` they need 64 KiB and do not. Your `cache_misses` chart shows L1 misses per FMA jumping from 0.17 to 1.1 between those sizes.

Blocking: arranging reuse to fit a cache

A cache keeps what you used recently. You cannot tell it what to keep, only order your accesses so that what you need again was used recently.

Blocking does that: work on a tile small enough to stay cached, and reuse it many times before moving on. In `multiplyBlock`, a `bs × bs` tile of B is reused for every row of the output tile. At `bs = 48` that tile is 18 KiB and stays in L1. At `bs = 96` it is 72 KiB and does not: that is the L1 cliff in your tile sweep, with 8× more L1 misses.

The payoff shows in your size sweep. Blocked holds about 7.4 GFLOP/s from `n = 100` to 4000, while naive falls from 6.6 to 1.2.

GPU side. Shared memory is a cache you manage by hand: your kernel loaded each tile explicitly and synchronized with `__syncthreads()` before using it. A CPU cache is managed by hardware, so you steer it only through access order. On recent NVIDIA GPUs, shared memory and L1 are carved from the same on-chip memory in each SM. There, a bigger tile cost shared-memory space and fewer resident blocks; here, it costs L1 and L2 space.

Associativity: why $n = 1024$ was $5\times$ slower

A cache is split into **sets**, and each address can live in only one set. Your L1 has 64 sets of 12 lines each (12-way). Address bits 6 to 11 pick the set, so addresses 4096 bytes apart always share a set.

At $n = 1024$ a row of B is 8192 bytes, so a column walk lands in the same set every time. Only 12 of the L1's 768 lines can hold it. These are **conflict misses**: the cache is mostly empty, but that one set is full.

The calibration runs showed it plainly. Blocked ran at 6.8 GFLOP/s with 0.02 L1 misses per FMA at $n = 1000$, and at 1.4 GFLOP/s with 1.0 misses per FMA at $n = 1024$. That is why the suite avoids powers of two; `TODO.md` holds a benchmark idea for studying the effect.

Prefetchers: why naive is not even slower

Hardware **prefetchers** watch for sequential or constant-stride access and fetch lines before you ask. Naive's column walk has a constant stride, so many of its L1 misses find the line already waiting in L2. That is how naive still makes 4 GFLOP/s at $n = 2000$ while missing L1 on 1.1 of every multiply-add.

Pages and the TLB

Addresses are virtual, and each 4 KiB page must be translated to a physical one. The **TLB** caches recent translations; a miss triggers a page-table walk, which is itself a chain of memory loads.

At $n = 4000$, naive's column walk strides 32,000 bytes, so almost every load lands on a new page. Inside a VM like WSL2 each walk is two-level: the guest's page tables live in memory that the host must translate too. That is the likely reason naive at $n = 3000$ and 4000 ran at two different speeds with identical miss counts.

Latency, bandwidth and Little's law

Latency is the time for one access; bandwidth is bytes per second. **Little's law** links them: bytes in flight = bandwidth $\times$ latency.

To stream 56 GB/s with the 110 ns DRAM latency measured here, about 6 KB, or roughly 95 lines, must be in flight at once. A core gets there only with many independent misses outstanding, supplied by prefetchers and out-of-order execution. Naive at $n = 4000$ runs at 1/20 of its bandwidth roof (section 1), so its misses are not overlapping: it is latency-bound, not bandwidth-bound.

GPU side. A GPU hides memory latency with the same law but a different source of parallelism: thousands of threads, each with a load in flight. Occupancy, the number of warps resident on an SM, is Little's law in practice.

To go deeper: CS:APP 6.2 (locality), 6.4 (sets and ways), 6.6 (the memory mountain, and loop orders for matrix multiply in 6.6.2), and 9.6 (the TLB). [Drepper, *What Every Programmer Should Know About Memory*](#) (2007), sections 3 and 4, is the classic long-form treatment. MIT 6.172 lecture 14, *Caching and Cache-Efficient Algorithms*, analyzes tiled matrix multiply formally.

3. Inside one core: latency, throughput and out-of-order execution

A modern core works on many instructions at once, so a loop's speed is set by its longest chain of dependent instructions, not by its instruction count. Your blocked loop is limited by a chain of adds. It runs as fast as it does only because the core overlaps neighboring chains.

Latency and throughput

CS:APP chapter 4 covers pipelining: an instruction passes through stages, and a new one can start before the last one finishes. That gives every instruction two numbers. **Latency** is the cycles until its result is usable. **Throughput** is how often a new one can start.

Measured on your core with dependent and independent chains of each instruction:

Instruction	Latency (cycles)	New ones started per cycle
Add, <code>vaddsd</code>	2	2
Fused multiply-add, scalar or 512-bit	4	2

A multiply, `vmulsd`, has a latency of 3. Notice the gap: an FMA takes 4 cycles to finish, but 2 can start every cycle. To keep both FMA units busy you need $4 \times 2 = 8$ independent FMAs in flight. The FMA microbenchmark shows it: with 4 independent accumulators it ran at 1.00 cycles per FMA (pure latency), with 6 at 0.69, with 8 at 0.56, and with 12 it reached the 0.50 limit.

Out-of-order execution

The core fetches several instructions per cycle into a window, the **reorder buffer**, which holds up to [448 instructions on Zen 5](#). It executes any instruction whose inputs are ready, in any order, then retires results in program order. Your blocked loop retires about 6.9 instructions per cycle (the IPC counter).

So a loop has two speed limits, as CS:APP 5.7 puts it. The **latency bound** is the longest chain of dependent operations. The **throughput bound** is the busiest execution unit. You hit whichever is slower.

Your loop's chain

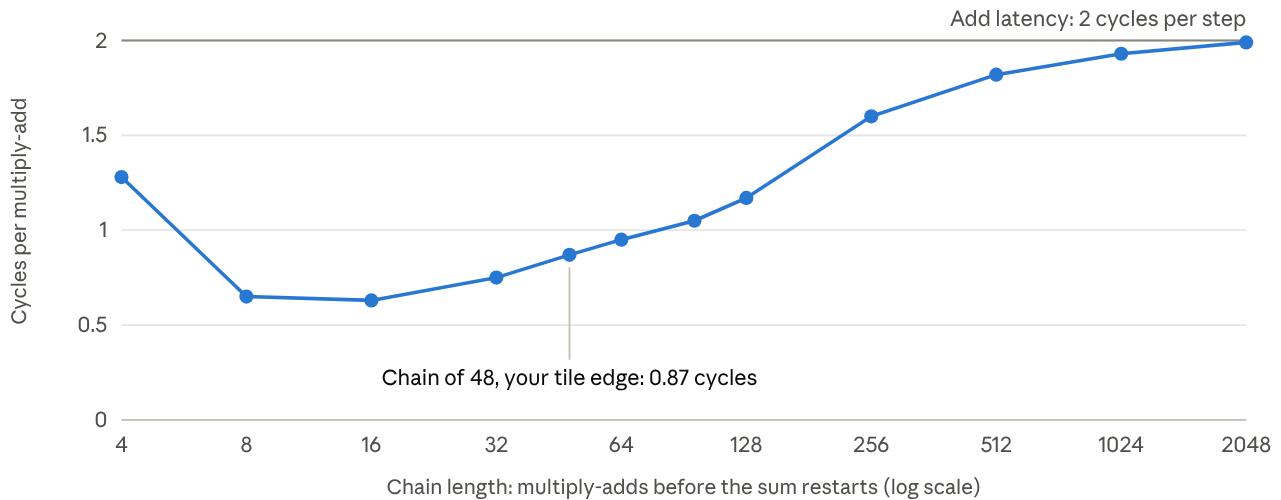
In the i-j-k loop, `sum += A[i][k] * B[k][j]` compiles to a multiply, then an add. Each multiply needs only loads, so multiplies run ahead. Each add needs the previous `sum`, so the adds form a chain at 2 cycles per step. At 5 GHz, one chain is 2.5 billion multiply-adds per second, or 5 GFLOP/s: the "one dependent add chain" ceiling in section 1.

Why not one FMA per step? GCC 13 predates Zen 5, so `-march=native` uses its Zen 4 tuning, which splits FMAs that sit in a dependency chain. A chain of 4-cycle FMAs would be twice as slow as a chain of 2-cycle adds. Forcing FMA anyway dropped the blocked kernel from 6.5 to 4.0 GFLOP/s, so the compiler was right. (Optimizations.md now says this; it used to claim an FMA chain.)

Yet blocked runs at about 1.4 cycles per step, above that 5 GFLOP/s ceiling. The reason is that `sum` restarts for every `(i, j)`. Each chain is only `bs` = 48 adds long, and the next chain does not depend on it. The out-of-order window holds the end of one chain and the start of the next, so they overlap. A synthetic loop with one chain per output shows the effect directly:

Short dependency chains overlap; long ones run at the add's 2-cycle latency

One add chain per output, like the k loop in multiplyBlock. Synthetic test on this 9800X3D.



synthetic test on your 9800X3D · cycle counter, 400 million multiply-adds per point

At 2048 steps per chain, the cost is the add latency, exactly 2.0 cycles. At 48, chains overlap and the cost falls to 0.87; at 4, loop overhead takes over. Your real loop does more work per step (the tail check, index arithmetic, strided loads), so it sits at 1.4, but the mechanism is the same. That is also why small tiles won the tile sweep: a short chain overlaps better.

The classic fixes

CS:APP 5.8 and 5.9 cover the standard cures for a latency-bound loop: unroll it, and give it several independent accumulators so chains run side by side. Both reorder floating-point additions, which changes the result in the last bits, so the compiler will not do it for you without `-ffast-math`. Optimizations.md asks guiding questions about a third fix: a loop order whose inner loop has no chain at all.

GPU side. A GPU core (an SM) hides latency differently. Each warp runs its instructions in order, with no reorder buffer; when a warp stalls on a dependent FMA, the scheduler issues from another warp the next cycle. So a GPU covers latency with more warps (occupancy), or with more independent work per thread, such as computing several outputs per thread. A CPU core has one or two threads and relies on its out-of-order window. Both obey the same rule: independent operations in flight must cover latency × issue rate.

To go deeper: CS:APP 4.4 (pipelining) and 5.7 to 5.9 (latency and throughput bounds, unrolling, multiple accumulators). MIT 6.172 lecture 4, *Assembly Language & Computer Architecture*, covers pipelines, superscalar issue and out-of-order execution. Stanford CS149 lecture 2, [A Modern Multi-Core Processor](#), explains instruction-level parallelism, SIMD and hardware multithreading in one framework that covers both CPUs and GPUs.

4. SIMD and FMA: the 32 FLOP per cycle you are not using

Your loop does one multiply-add per 1.4 cycles on one double. The core could start two per cycle on 8 doubles each. That is $2.8\times$ from the step rate (the chain and loop overhead) and $8\times$ from SIMD width: about $22\times$ in total, which matches the measured 1.47 FLOP per cycle against a peak of 32.

Vector registers and instruction names

SIMD (single instruction, multiple data) instructions apply one operation to every lane of a wide register. With AVX-512, a `zmm` register holds 512 bits, or 8 doubles, and one `vfmadd231pd` does 8 multiply-adds.

The instruction names tell you what the compiler did:

- `sd` = scalar double, one value. `pd` = packed double, a full register.
- `xmm` = 128 bits (2 doubles), `ymm` = 256 bits (4), `zmm` = 512 bits (8).
- `vfmadd...` = fused multiply-add; `vmulsd` then `vaddsd` = a separate multiply and add.

Here is the inner `k` loop of `multiplyBlock` as compiled today, from `objdump -d build/matmul.o` (AT&T syntax, as in CS:APP: source first, destination last):

```
inc    %rax                # k++
add    %rcx,%rdx           # point at the next row of B: +n*8 bytes
cmp    %rsi,%rax           # the tail check: k reached A.cols?
jae    <exit>              # if so, break
vmovsd (%r15,%rax,8),%xmm0 # load A[i][k], one double
vmulsd (%rdx),%xmm0,%xmm0  # multiply by B[k][j], one double
vaddsd %xmm0,%xmm1,%xmm1   # sum += product: needs the previous sum
cmp    %rdi,%rax           # end of this K block?
jne    <top>               # if not, loop
```

Nine instructions for one multiply-add, all scalar. Only the `vaddsd` is on the chain; the rest is overhead that the out-of-order core hides.

Why the compiler will not vectorize this loop

1. **Floating-point addition is not associative.** $(a + b) + c$ and $a + (b + c)$ round differently. Vectorizing the sum over k means 8 partial sums added at the end: a different order, so a slightly different answer. The compiler must give the exact result your code specifies unless you allow reordering with `-ffast-math` or `-fassociative-math`. This is also why your tests compare naive and blocked results with a tolerance.
2. **The access is strided.** `B[k][j]` walks down a column, so consecutive k values are a whole row apart. A vector load wants 8 neighbors in memory. Gather instructions exist, but they are slow.
3. **The early break.** A loop with a data-dependent exit has no trip count known in advance, which blocks vectorization. `Optimizations.md` has a hint about this.

To see the compiler's own reasons, rebuild `src/matmul.cpp` with `-fopt-info-vec-missed`.

Fused multiply-add

An FMA computes $a * b + c$ with a single rounding, so it is both faster and slightly more accurate than a separate multiply and add. Peak FLOP numbers assume every slot does an FMA. Its one weakness is latency: 4 cycles, against 2 for an add. In a dependency chain that matters, which is why GCC avoided FMA in your loop. In a loop with no chain, FMA is pure gain.

GPU side. A warp is 32 threads running the same instruction in lockstep. NVIDIA calls it SIMT; it is SIMD where each lane has its own address and can be switched off when its branch is not taken. Your CUDA threads looked scalar, but the hardware ran them 32 wide. The same rules apply on both sides:

- Contiguity. When the 32 threads of a warp read consecutive addresses, the loads coalesce into a few memory transactions; strided loads split into many. That is the CPU's contiguous vector load versus a gather.
- Uniform control flow. A divergent branch masks off lanes, and AVX-512 has mask registers for exactly the same job.

The difference is who writes the SIMD: on a GPU the programming model creates it for you, while on a CPU you or the compiler must.

To go deeper: CS:APP 2.4 explains floating-point rounding, and Goldberg's paper *What Every Computer Scientist Should Know About Floating-Point Arithmetic* (ACM Computing Surveys, 1991) is the classic. [Algorithms for Modern Hardware](#) by Sergey Slotin has an

excellent chapter on SIMD parallelism with x86 intrinsics. Hennessy and Patterson chapter 4 compares vector, SIMD and GPU designs. Intel's *Intrinsics Guide*, a searchable reference, is where you will look up `_mm512_fmadd_pd` and friends.

5. Many cores: scaling, SMT and scheduling

Your pool turns 8 workers into a 6.4× speedup and 16 workers into 8.0×. The shortfall at 8 comes from fewer instructions per cycle per thread, not from clock speed, and the gain past 8 comes from SMT.

Speedup, efficiency and Amdahl's law

Your scaling benchmark is **strong scaling**: a fixed problem ($n = 2000$) split among more workers. **Weak scaling** grows the problem with the workers; the suite does not measure it.

- **Speedup** $S(p) = T(1) \div T(p)$. **Efficiency** $E(p) = S(p) \div p$.
- Which $T(1)$? *Absolute* speedup uses the best serial code, `multiplyBlocked`; *relative* speedup uses the parallel code on one worker. The suite reports absolute speedup. Yours barely differ: one worker took 2,143 ms against 2,181 for the serial code, so the pool itself costs nothing at this task size.
- **Amdahl's law**: if a fraction f of the work parallelizes, $S(p) \leq 1 \div ((1 - f) + f \div p)$. With 5% serial work, no number of cores beats 20×. Your serial part (allocating C , enqueueing 1,764 tiles) is tiny, so Amdahl is not your limit yet.

A warning worth remembering: an inefficient parallel program can scale beautifully. McSherry, Isard and Murray, [Scalability! But at what COST?](#) (HotOS 2015), found big-data systems that needed hundreds of cores to beat one well-written thread. That is why the suite compares against the best serial code.

What held 8 workers to 6.4×

The counters rule out the usual suspects and point to the cause:

- **Not the clock.** The cores ran at 5.07 GHz with 8 workers and 5.08 with one.
- **Not DRAM bandwidth.** Blocked pulls about one DRAM line per thousand multiply-adds.
- **Fewer instructions per cycle.** Per-thread IPC fell from 6.87 to 5.64 (−18%), and L1 misses per FMA doubled. Then $8 \times 5.64 \div 6.87 \approx 6.6$, which accounts for nearly all of the shortfall.

The likely cause is placement. Under WSL2, Hyper-V decides which physical core runs each virtual CPU, and pinning threads inside the VM changed nothing in a test. When two workers land on one core, they share its 48 KiB L1 and its execution units, which is exactly the pattern above.

SMT: 16 workers on 8 cores

Simultaneous multithreading (SMT; Intel calls it Hyper-Threading) runs two hardware threads on each core. They have separate registers but share the execution units, the caches and the out-of-order machinery.

SMT helps when one thread leaves the core idle, and your kernel does: while one thread waits on its add chain, the other can issue. Your numbers:

- 16 workers ran at 58.7 GFLOP/s against 47.2 for 8: **+24%**.
- Per-thread IPC fell to 3.8, but each core retired about 7.6 instructions per cycle, against 5.64 with one thread.
- The clock dropped to 4.66 GHz, because every core was doubly busy.
- L1 misses per FMA rose to 0.057, since two threads now share 48 KiB.

Two habits follow. Call it "8 cores, 16 threads", never "16 cores", and compute efficiency against the 8 physical cores. Expect the SMT gain to shrink once your kernel saturates the FMA units, but measure it: OpenBLAS also gained 30% from 16 threads in a quiet test.

Static vs dynamic scheduling: your pool vs OpenBLAS

Your pool uses **dynamic scheduling**: 1,764 tile tasks sit in one queue, and each worker takes the next one when it finishes. OpenBLAS uses **static scheduling**: it splits the matrix into one piece per thread up front.

Dynamic scheduling balances itself. If one thread is slowed by a shared core or a Windows interrupt, the others take more tiles. With a static split, everyone waits for the slowest piece. Your data shows the difference twice:

- At 8 threads your pool reached 6.4× its one-thread speed; OpenBLAS reached 4.3×.
- At 16 threads your repetitions varied by 2% (CV); OpenBLAS's varied by 33%.

The price of dynamic scheduling is per-task overhead: every enqueue and dequeue takes the pool's mutex. Your tile sweep shows it at the small end. Tile 8 means 62,500 tasks, and it lost 31% against tile 48 with 8 workers, but only 13% single-threaded. Work stealing, used by Cilk and TBB, cuts that cost with a queue per worker; MIT 6.172 lecture 13 covers it.

Bandwidth is shared too

One core reads 56 GB/s from DRAM, and all 16 threads together reach only 58 GB/s. A memory-bound kernel cannot scale with cores on this machine. Matmul scales only because blocking keeps it compute-bound.

False sharing

Caches stay coherent in units of 64-byte lines. If two cores write different variables that share a line, the line bounces between their caches on every write. Your tiles write disjoint parts of C, so at most the lines on tile edges are shared, and each is written only once per K block. It is a classic cause of mysterious slowdowns, so keep it in mind for future designs.

GPU side. A GPU's block scheduler hands out thread blocks to SMs as they free up, the same dynamic scheduling as your queue. A grid with too few blocks leaves SMs idle at the end, like 16 tiles on 12 workers. And where a CPU core runs two hardware threads, an SM keeps dozens of warps resident and switches between them every cycle. That is SMT taken to its extreme, and it is the GPU's main way of hiding latency.

To go deeper: CS:APP 1.9.1 (Amdahl's law) and 12.6 (speedup, efficiency, strong and weak scaling with threads). MIT 6.172 lectures 6, 8 and 13 cover multicore programming, work and span, and the Cilk work-stealing scheduler. Stanford CS149 lecture 5, *Work Distribution and Scheduling*, compares static and dynamic assignment. For the theory of concurrent data structures, such as a better queue for your pool, see Herlihy and Shavit, *The Art of Multiprocessor Programming*.

6. Measuring without fooling yourself

A benchmark is an experiment, and most benchmarking mistakes are experimental-design mistakes: timing the wrong thing, letting the compiler skip the work, or mistaking noise for an effect. Every design choice in the suite guards against one of them.

Design decisions, and the mistake each one prevents

Decision in the suite	What goes wrong without it
Wall-clock time (<code>UseRealTime()</code>)	The calling thread sleeps in <code>waitAll()</code> , so its CPU time leaves out the workers and the pool looks many times faster than it is.

Decision in the suite	What goes wrong without it
Inputs and the pool built outside the timed loop	You time <code>randomMatrix</code> or thread creation instead of the multiply.
Result allocation inside the timed call	You hide a cost callers really pay: 32 MB allocated and zeroed per call at $n = 2000$.
<code>benchmark::DoNotOptimize(C)</code>	The compiler may delete work whose result is never used, and you time nothing.
Every result checked after timing	A fast but wrong kernel posts a number.
Fixed random seeds	Runs multiply different matrices and stop being comparable.
No power-of-two sizes	One pathology, cache-set conflicts, dominates every number.
Rebuild when a header changes (<code>-MMD -MP</code>)	You benchmark stale code without noticing.
Compiler, flags, commit and CPU saved in each results file	Nobody, including you next month, can tell what a number measured.

Freivalds' check: verifying a product in $O(n^2)$

Recomputing $C = AB$ to check it costs $O(n^3)$, as much as the benchmark. Freivalds' algorithm (1977) is cheaper: pick a random vector x and compare $A(Bx)$ with Cx . That is three matrix-vector products, $O(n^2)$.

If C is wrong, then $D = C - AB$ has a nonzero entry, and Dx is almost never zero for a random x . With x drawn from $\{0, 1\}^n$, one trial catches a wrong C with probability at least $1/2$. The suite draws real numbers, for which a wrong C essentially never slips through.

Floating point adds a twist: $A(Bx)$ and Cx differ by rounding even when C is right. So the suite allows twice the worst-case rounding error, which grows with the number of terms summed. A real bug, like a missing k term or a skipped tile, is off by many orders of magnitude more. In testing, both of those were caught, and correct products summed in a different order passed. Mitzenmacher and Upfal, *Probability and Computing*, section 1.3, proves the probability bound.

Where noise comes from

- **The clock.** Boost and power limits change the frequency from run to run. The GHz counter exposes it: one slow repetition in your run ran at 4.4 GHz instead of 5.1.
- **Other programs.** Windows background work (Defender, OneDrive syncing this folder) and the WSL2 VM itself.
- **Memory layout and placement.** Where pages land in physical memory, and which core a thread runs on. Mytkowicz et al., [Producing Wrong Data Without Doing Anything Obviously Wrong!](#) (ASPLOS 2009), showed that innocent setup changes, like the size of the UNIX environment or the link order, shift results enough to flip conclusions.

In your run, single-threaded blocked varied by about 1%, naive at $n = 3000$ and 4000 ran at two distinct speeds, and OpenBLAS at 16 threads varied by 33%.

Repetitions, medians and the coefficient of variation

- **Repeat.** `make bench-run` runs each benchmark 5 times.
- **Summarize with the median.** Performance data is rarely a bell curve: noise only ever adds time, so the distribution is skewed, and sometimes it has two peaks, like naive's. A slow outlier drags the mean but not the median. In Hoefler and Belli's words, normal distributions are "only rarely observed when measuring computer performance".
- **The minimum** is defensible for purely CPU-bound code, since noise only slows runs down, but it rewards lucky runs. The suite's charts show the median with bars from slowest to fastest.
- **Summarize times, then derive rates.** The suite computes GFLOP/s from the median time rather than averaging GFLOP/s across runs; averaging rates or ratios with an arithmetic mean gives wrong answers (Hoefler and Belli, rules 3 and 4).
- **The CV**, standard deviation $\div$ mean, is the noise as a fraction of the signal. Treat a difference smaller than about twice the CV as noise.
- **Interleave.** The suite runs repetitions in a random order across all benchmarks, so slow drift, like heat building over 20 minutes, spreads over everything instead of landing on one benchmark.

Is a difference real? Hypothesis tests from scratch

Suppose you run the old and new kernel 9 times each, and the new median is 3% faster. Could a gap that size appear by chance even if they were equally fast?

- **The null hypothesis** says both are equally fast. **The p-value** is the probability of a difference at least this large if the null hypothesis were true. A small p-value, conventionally under 0.05, makes chance an unlikely explanation. It is not the probability that the new kernel is faster.
- **The Mann-Whitney U test**, which `compare.py` runs, works on ranks instead of raw times. Over all 81 (old, new) pairs, count how often the new run was faster: that count is U. If the kernels are equally fast, U lands near 40.5; if every new run beats every old one, U = 81. The test turns U into a p-value.
- **Why ranks.** The test assumes no bell curve, which matters for skewed or two-peaked timing data. Hoefler and Belli's rule 6: do not assume normality without checking.
- **Why at least 9 repetitions.** With 3 runs each, the 6 ranks can split in only 20 equally likely ways, so even a perfect separation gives $p = 2/20 = 0.10$. No effect, however large, can reach significance; `compare.py` warns below 9.
- **Significance is not importance.** With 100 runs, a 0.5% difference can be significant and still not matter. Report the size of the difference and its spread, with the p-value as support.

When `compare.py` compared the same code from two separate runs, it reported $p = 0.79$: no evidence of a difference, which is the right answer.

Reporting results honestly

- Show the spread, not just one number.
- State the baseline: speedup against the best serial code, with that code's absolute time (Hoefler and Belli, rule 1).
- Give the context: CPU, compiler, flags, commit and OS. Each results file records them.
- Choose axes on purpose: log scales for ratios across orders of magnitude, linear scales when absolute differences matter, and never two y-axes on one chart.
- Compare against a strong reference, as the suite does with OpenBLAS.

[Hoefler and Belli, *Scientific Benchmarking of Parallel Computing Systems*](#) (SC '15) gathers twelve such rules. Read it before you publish a number.

GPU side. GPU timing has the same traps and one more: kernel launches are asynchronous. A CPU timer around a launch measures only the launch, unless you synchronize first or time with CUDA events. The first launch also pays one-time setup costs, so warm up before measuring, and GPU clocks boost and throttle just like CPU clocks.

To go deeper: MIT 6.172 lecture 10, *Measurement and Timing*, is the best single lecture on this topic. [Georges, Buytaert and Eeckhout, Statistically Rigorous Java Performance Evaluation](#) (OOPSLA 2007) applies these statistics to performance data, and its lessons carry over from Java. Raj Jain's *The Art of Computer Systems Performance Analysis* is the classic textbook on experimental design. For statistics basics, any introductory text works; *OpenIntro Statistics* is free.

7. Hardware performance counters

A timer tells you how fast; hardware counters tell you why. They count events inside the CPU, like cycles, instructions and cache misses. The suite reads six of them around every timed loop and divides by the number of multiply-adds.

What a PMU is

Each core has a **performance monitoring unit** (PMU): a few registers that you program with an event code and that count while your code runs. Linux exposes them through the `perf_event_open` system call, which the `perf` tool wraps.

The suite's `bench/perf_counters.cpp` does four things:

1. Opens one counter per event, stopped, and asks for user-space counts only, which Linux allows by default.
2. Sets `inherit`, so threads created later count too. That is why counters are opened before the pool: its workers inherit them.
3. Enables the counters right before the timed loop and disables them right after.
4. Scales each count by $\text{time-enabled} \div \text{time-running}$, in case the kernel had to time-share counters among more events than the hardware has.

The events and what they mean

Counter	What it is	Your blocked kernel at n = 2000
GHz	$\text{cycles} \div \text{CPU seconds}$: the real clock while your code ran	5.0
IPC	$\text{instructions} \div \text{cycles}$, per hardware thread	6.9

Counter	What it is	Your blocked kernel at n = 2000
FLOP/cycle	$2 \times \text{multiply-adds} \div \text{cycles}$ (single-threaded runs only); the peak is 32	1.47
L1D_miss/FMA	L1 data-cache read misses per multiply-add	0.013
L2_miss/FMA	requests that missed L2 (AMD event 0x64)	0.003
L3_fill/FMA	lines brought into L1 from L3 (AMD event 0x44, unit mask 0x02)	0.003
DRAM_fill/FMA	lines brought into L1 from DRAM (event 0x44, mask 0x08)	0.0005

Calibrate a counter before you trust it

Event names are slippery. The generic "cache-misses" event means last-level cache misses on Intel but L2 misses on AMD. Events change between CPU generations, and some do not work inside virtual machines at all.

So each event was checked against a test with a known answer: a random pointer chase over a growing working set. Each counter should switch on exactly when the data outgrows the cache level above it:

Working set	L1D miss / load	L2 miss / load	L3 fill / load	DRAM fill / load	Cycles / load
16 KiB	0.00	0.00	0.00	0.00	4
256 KiB	1.00	0.00	0.00	0.00	14
4 MiB	1.00	0.92	0.93	0.00	57
32 MiB	1.00	1.02	1.00	0.01	98
512 MiB	1.00	2.50	1.48	0.99	551

Each level switches on where it should. The counts above 1 at 512 MiB come from TLB misses: the page-table walks miss in cache too.

Normalize per unit of work

Raw counts grow with the problem, so divide by the work done. Per multiply-add makes naive at $n = 500$ comparable with blocked at $n = 4000$. Misses per thousand instructions (MPKI) is the more common unit, but it misleads here: naive and blocked run different numbers of instructions per multiply-add.

Pitfalls

- **A miss rate is not a cost.** It matters only if the core waits on those misses. Naive at $n = 2000$ misses L1 on every multiply-add yet still makes 4 GFLOP/s, because prefetchers and the out-of-order window hide most of it.
- **Counting is per thread.** OpenBLAS's threads exist before the counters, so its multithreaded runs report none.
- **The fill counters undercount memory traffic.** They count lines arriving in L1. When the L2 prefetcher fetches a line from DRAM first, the L1 fill comes from L2. Use them to compare kernels, not as exact byte counts; that is why the roofline in section 1 says "estimated".

The next step beyond counting is sampling: interrupting every N events and recording which instruction was running, to find out *where* the misses happen. The `perf record` command does this, and top-down analysis builds on it to split cycles into front-end, back-end, bad speculation and retiring.

GPU side. Nsight Compute is the GPU's counter tool. It reports achieved occupancy, memory throughput, cache hit rates and warp stall reasons, and it draws the roofline from section 1. The same discipline applies: know what each metric counts, normalize per unit of work, and check the stall reasons before you optimize a miss rate.

To go deeper: Denis Bakhvalov's [Performance Analysis and Tuning on Modern CPUs](#) is free, practical and current; read its chapters on performance monitoring and top-down analysis. Brendan Gregg's *Systems Performance* (2nd edition) covers the `perf` tool in depth. The `perf_event_open(2)` man page documents every field the suite sets.

8. Reading your results

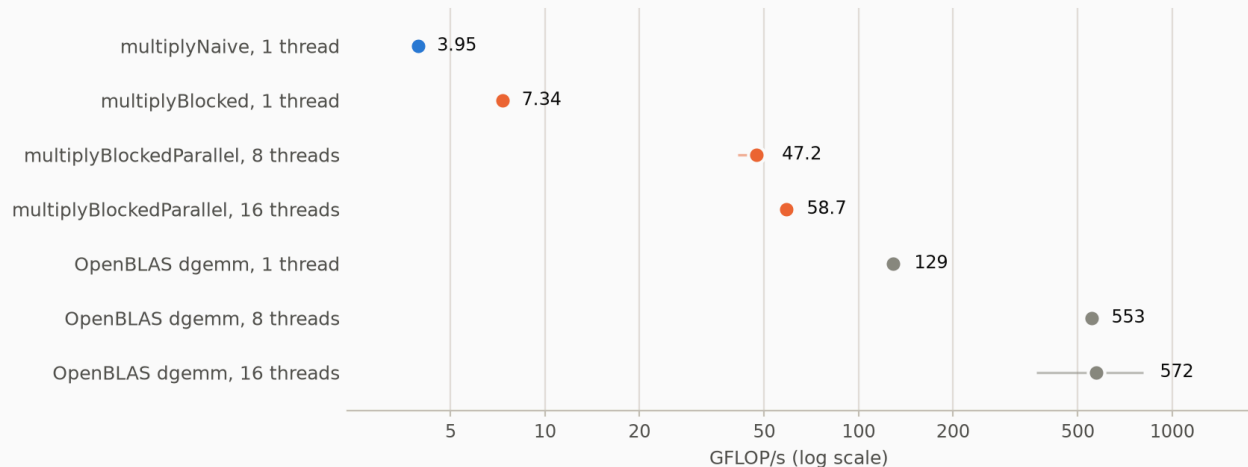
The first full run says your blocked kernel holds about 7.4 GFLOP/s on one core at every size, reaches 59 GFLOP/s on 16 threads, and runs at 5.7% of OpenBLAS on one core. Each chart in `bench/plots` shows one piece of that. Treat the numbers as preliminary: they come from one noisy session, so re-run on a quiet machine before you quote them.

The headline: where you stand

Matrix multiply throughput at $n = 2000$

GFLOP/s on a log scale: median of 5 runs, bars span the slowest to fastest run

• Naive • Blocked, yours • OpenBLAS reference



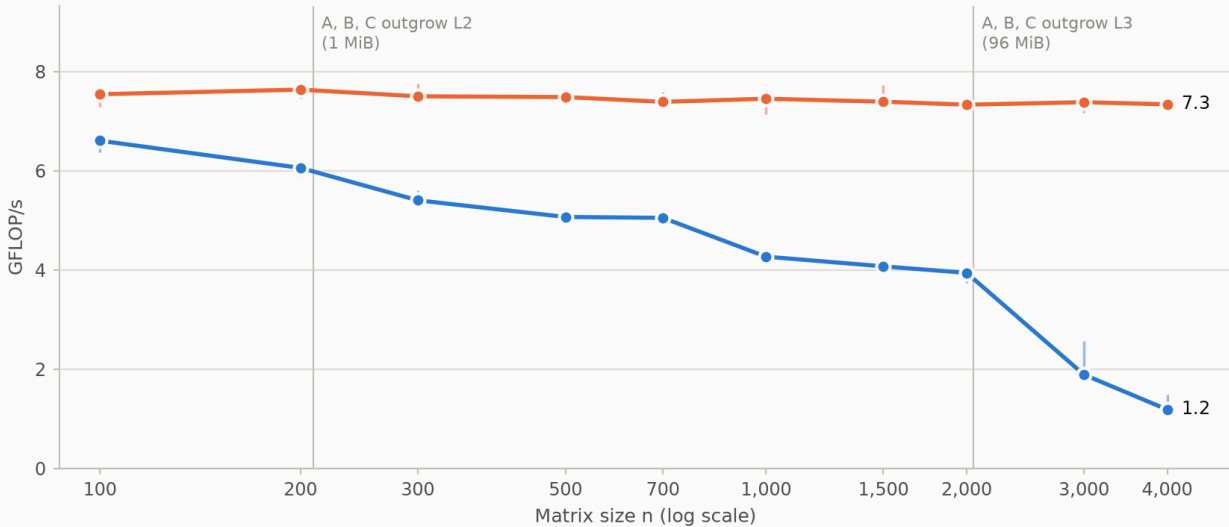
- Read it as a ladder: naive 3.95 GFLOP/s, blocked 7.34 (1.9×), blocked with 8 workers 47.2 (12× naive), with 16 workers 58.7 (15×).
- OpenBLAS does 129 on one thread, 553 on 8 and 572 on 16. The log axis hides how big that gap is: OpenBLAS on one core beats your 16-thread version by 2.2×.
- The wide bar on OpenBLAS at 16 threads is real spread, from 19.8 to 43.4 ms per run. It is the noisiest benchmark in the suite, for the reasons in section 5.

Naive vs blocked across sizes

Naive vs cache-blocked multiply, one thread

GFLOP/s for $n \times n$ doubles (tile 48): median of 5 runs, bars span the slowest to fastest run

—●— multiplyNaive —●— multiplyBlocked



- **Blocked is flat**, 7.3 to 7.6 GFLOP/s from $n = 100$ to 4000. Flat is the signature of good blocking: the data the inner loops touch fits in L1 whatever the matrix size.
- **Naive falls in three steps**, each where a working set outgrows a cache (section 2):
 - 6.1 to 5.1 GFLOP/s between $n = 200$ and 500, as all of B outgrows the 1 MiB L2 (at $n \approx 362$);
 - 5.1 to 4.3 between $n = 700$ and 1000, as one column of B outgrows the 48 KiB L1 (at $n = 768$);
 - 4.0 to 1.9 and 1.2 at $n = 3000$ and 4000, as the three matrices outgrow the 96 MiB L3 (at $n \approx 2048$).
- **The ratio grows with size**: blocked is $1.14\times$ faster at $n = 100$, $1.9\times$ at 2000 and $6.2\times$ at 4000.

Why only $1.9\times$ at $n = 2000$? Both kernels are limited by the add chain (section 3), and naive's misses mostly hit L2 or the 96 MiB L3. The 3D V-Cache is doing naive a big favor. On a Ryzen 7 9700X, with 32 MiB of L3, the matrices would outgrow L3 near $n = 1200$ and naive would hit its cliff much sooner.

Where the data came from

Where each multiply-add's data came from

Hardware-counter events per FMA, one thread, by matrix size n (log scales). Lines: where naive's working set outgrows a cache.

—●— multiplyNaive —●— multiplyBlocked



Read the three panels at one size. At $n = 2000$, naive misses L1 on 1.09 of every multiply-add against 0.013 for blocked, 82 $\times$ more. It misses L2 23 $\times$ more often. Yet DRAM traffic is tiny for both, under one line per thousand multiply-adds, because the L3 absorbs it.

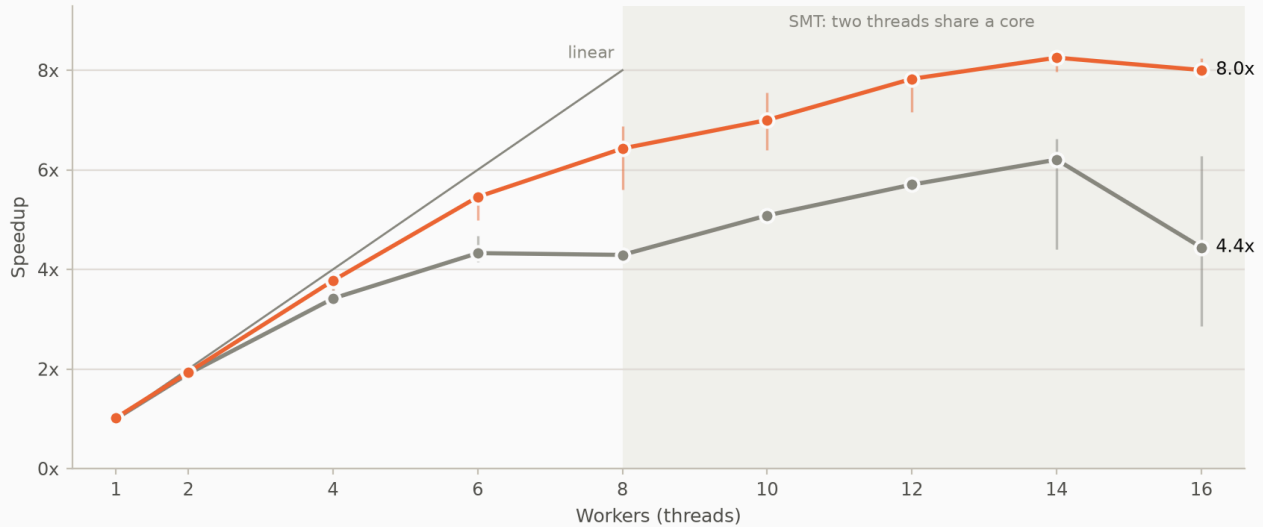
Past $n = 2048$ the matrices no longer fit in L3. Naive's DRAM fills rise 380 $\times$, to 0.076 per multiply-add at $n = 4000$, while blocked's stay near 0.001. That is the cliff in the previous chart, explained event by event. This is the point of counters: you can name the bottleneck instead of guessing it.

Thread scaling

Speedup from more workers

$n = 2000$, against each implementation's own one-thread median; bars span the slowest to fastest run

—●— multiplyBlockedParallel (your pool) —●— OpenBLAS



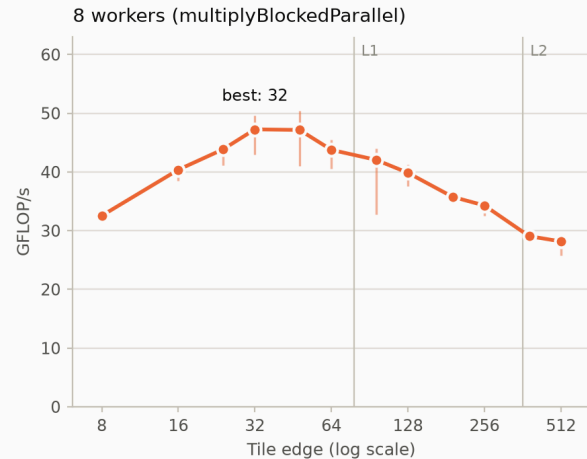
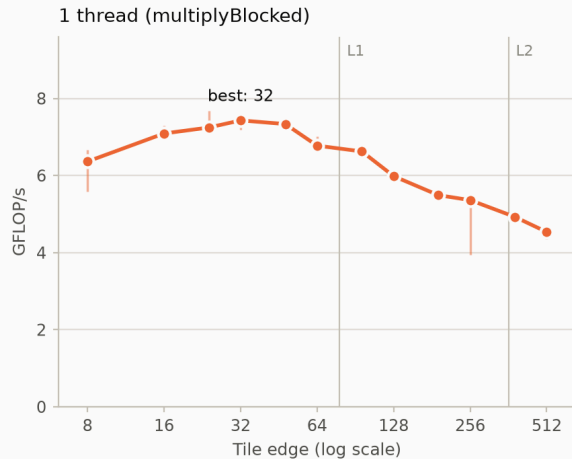
- **Near-linear to 6 workers:** 5.5× at 6, 91% efficiency.
- **Bending at 8:** 6.4×, 80%. The counters blame lower IPC per thread, not the clock (section 5).
- **SMT pays past 8:** 7.0× at 10, 7.8× at 12, 8.25× at 14, 8.0× at 16, with the clock falling to 4.66 GHz at 16.
- **The bars show the spread.** Your pool's runs vary by 2 to 8%; OpenBLAS's swing widely at 14 and 16 threads.

So, do 16 workers help on 8 cores? For this kernel, yes: +24% over 8, because it is latency-bound and SMT fills the idle slots. Re-test after you vectorize.

Tile sweep

Tile size sweep

GFLOP/s by tile edge at $n = 2000$, bars span the slowest to fastest run. Lines: where one tile of B ($\text{edge}^2 \times 8$ bytes) fills L1 and L2.



- **A broad optimum.** Single-threaded, every tile from 16 to 48 is within 5% (7.09 to 7.43 GFLOP/s), and 32 and 48 tie.
- **Two cliffs** where one tile of B outgrows a cache: L1 between 64 and 96 (L1 misses per FMA jump from 0.018 to 0.14), and L2 between 256 and 384 (L2 misses jump from 0.0015 to 0.047).
- **A slope in between.** Bigger tiles mean longer add chains, which overlap less: IPC falls from 7.06 at tile 16 to 4.78 at tile 256 (section 3).
- **Small tiles hurt more in parallel.** Tile 8 means 62,500 tasks through one mutex; it loses 31% with 8 workers but 13% single-threaded.

Re-run the sweep after every kernel change. The best tile depends on the kernel: a vectorized kernel will want tiles shaped by its register blocking as well as by L1 and L2.

9. What comes next: the optimization ladder

Cache blocking was the first rung of a well-known ladder, and the big gains are on the next rungs: removing the dependency chain, SIMD, register blocking and packing. This section describes each rung as a concept and says what your benchmarks should show when you climb it. The code is yours to write; Optimizations.md has guiding questions.

Rung 1: remove the loop-carried chain

Your inner loop sums into one `sum`, so every step waits on the previous add (section 3). Some loop orders have an inner loop whose iterations are independent: each one updates a different element.

What to expect: the one-chain ceiling from section 1 disappears. The tile sweep's slope, caused by chain length, should flatten. And SMT should help less, because one thread can now keep the core busy.

Rung 2: SIMD and FMA

Once the inner loop walks contiguous memory and has no chain, each step can become one 8-wide FMA, whether written by the compiler or by you with intrinsics. Check the instructions (section 4): `vfmadd...pd` on `zmm` registers is the goal.

What to expect: `FLOP/cycle` rises from 1.5 toward the 32 peak, though not all the way. The next limit is loads and stores, because a simple vector loop loads and stores C for every FMA.

Rung 3: register blocking

Keep a small block of C in registers for the whole `k` loop, and let each loaded value of A and B feed several FMAs. This is the **micro-kernel** at the heart of every fast matmul. It is the same idea as cache blocking, one level up: registers are the fastest, smallest memory.

What to expect: more FLOPs per load, a large jump in `FLOP/cycle`, and a new reason for the tile sweep to change shape.

Rung 4: packing

Copy the current blocks of A and B into contiguous buffers, in exactly the order the micro-kernel reads them. Every later access becomes sequential: no strides, few TLB misses, and no cache-set conflicts.

What to expect: power-of-two sizes stop being pathological, and large `n` stops depending on the 96 MiB L3. The copy costs $O(n^2)$, which is small against $O(n^3)$ work.

Rung 5: parallelize the right loop

With fast kernels, the threading design matters more. Which loop you split across threads decides which data each core shares, and pool overheads and the 32 MB result allocation (`TODO.md`) become visible shares of the time.

How OpenBLAS reaches 25 FLOP per cycle

OpenBLAS follows [Goto and van de Geijn, *Anatomy of High-Performance Matrix Multiplication*](#) (ACM TOMS, 2008). The structure is five nested loops around a hand-written assembly micro-kernel. A block of A is packed to fit in L2 and a panel of B to fit in L3, and the micro-kernel keeps its block of C in vector registers, issuing FMAs back to back. That is every rung above, tuned for each CPU: it detected your Zen 5 as Cooperlake, an AVX-512 kernel.

GPU side. CUDA matmul climbs the same ladder. Si Boehm's [How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance](#) (2022) measures each step on one GPU:

1. Naive: 1.3% of cuBLAS.
2. Coalesced global memory: 8.5%.
3. Shared-memory tiling, the version you wrote: 12.8%.
4. Register tiling, each thread computing a column of outputs, then a 2D block: 36.5%, then 68.7%.
5. Vectorized loads, autotuning and warp tiling: up to 93.7%.

The lesson matches your CPU results exactly. Tiling for the shared cache is necessary but small; the large gains come from register blocking, which turns each load into many FMAs.

To go deeper: [LAFF-On Programming for High Performance](#) (van de Geijn, Myers and Parikh, free) walks a CPU matmul up this exact ladder in four weeks: loop orders, register blocking, packing, then OpenMP. MIT 6.172 lecture 1 shows a similar ladder in one hour. Slotin's [Algorithms for Modern Hardware](#) has a matrix multiplication case study that climbs the same ladder.

10. Exercises

Each exercise follows the same loop: write down a prediction with a number, measure, then explain the gap with a counter. A wrong prediction you can explain teaches more than a right guess.

- ☐ **Measure your noise floor.** Run the same code twice: `make bench-run LABEL=a ARGS=-benchmark_filter=BM_Blocked/n:2000 BENCH_REPS=9`, then again with `LABEL=b`, then `make bench-compare BASE=a LABEL=b`.
 - Predict: how many of the 12 benchmarks will show $p < 0.05$?

- Explain: with many tests, about 1 in 20 will look significant by pure chance. That is the multiple-comparisons problem, and it is why one surprising p-value is not a finding.
- ☐ **Break the timing on purpose.** Remove `->UseRealTime()` from `BM_BlockedParallel`, rebuild, and run it with 16 workers.
 - Predict the reported time and GFLOP/s before you run it.
 - Explain what the main thread was doing, then restore the line.
- ☐ **Re-run the FMA experiment.** Rebuild with FMA chains allowed: `make clean && make bench OPTFLAGS='-O3 -march=native -g -mtune-ctrl=^avoid_fma256_chains'` (clean first, because changed flags do not trigger a rebuild).
 - Predict the change in blocked GFLOP/s from the latencies in section 3, then compare it with `make bench-compare`.
 - Repeat after you remove the chain (rung 1). Predict whether FMA helps then.
- ☐ **Explain power-of-two sizes.** Add the benchmark from `TODO.md` ($n = 1000, 1024$ and 1048 , with counters).
 - Predict naive's and blocked's L1 misses per FMA at $n = 1024$, using the set arithmetic in section 2.
 - Predict again for blocked after you add packing (rung 4).
- ☐ **Count instructions per multiply-add.** After each kernel change, find the inner loop with `objdump -d -C --no-show-raw-insn build/matmul.o`.
 - Today it is 9 scalar instructions per multiply-add. Predict the count for your next version, then check it against the `IPC` and `FLOP/cycle` counters.
- ☐ **Place each new kernel on the roofline.** Compute its arithmetic intensity from `DRAM_fill/FMA` at $n = 4000$ (64 bytes per fill, 2 FLOPs per multiply-add).
 - Predict which line it sits under before you plot it: memory, peak, or one of the ceilings.
- ☐ **Watch SMT change.** After each rung, compare the 8- and 16-worker results.
 - Predict when the SMT gain will shrink below 5%, and which counter will tell you why.

11. Interview prep

Interviewers rarely care how fast your kernel is; they care whether you know why it is that fast and how you know. Lead with the method and the explanation, then the numbers. Re-run the suite on a quiet machine before you quote any of them.

A 60-second pitch

I built a thread pool in C++ and used it to parallelize a cache-blocked matrix multiply, then wrote a benchmark suite to understand the performance rather than just time it. It uses Google Benchmark with hardware performance counters read through `perf_event_open`, checks every result with Freivalds' $O(n^2)$ algorithm, and compares against OpenBLAS. Blocking made the kernel independent of matrix size, 1.9× faster than naive at $n = 2000$ and 6× at $n = 4000$, and 16 threads reached 15× naive. The counters showed the single-thread kernel is limited by a dependency chain of floating-point adds, not by memory, which tells me the next step is SIMD and register blocking, where OpenBLAS gets its 25 FLOP per cycle against my 1.5.

Likely questions, and strong answers

- **Why is blocked faster than naive?** Each tile is reused many times while it sits in L1. The counters show 82× fewer L1 misses at $n = 2000$. Then say why it is *only* 1.9× there: both kernels are latency-bound on the add chain, and the 96 MiB L3 catches most of naive's misses. Past $n = 2048$ the gap grows to 6×.
- **Why does 8 workers give only 6.4×?** The clock held at 5.07 GHz and DRAM traffic was tiny, so neither was the limit. Per-thread IPC fell 18% and L1 misses doubled, which points to threads sharing cores; under WSL2 you cannot control placement.
- **Does hyperthreading help?** For this kernel, yes, +24%, because it is latency-bound and the second thread fills idle issue slots. Once the kernel saturates the FMA units it should help less, but OpenBLAS also gained in a quiet test, so I would measure rather than assume.
- **How do you know your numbers are reliable?** Wall-clock time, inputs and threads set up outside the timed loop, 5 interleaved repetitions, medians with the CV reported, and counters to explain outliers, such as one run at 4.4 GHz. For A/B changes I use a Mann-Whitney U test with at least 9 repetitions.
- **Why the median, not the mean?** Timing data is skewed and sometimes has two peaks, so one slow run drags the mean. Naive at $n = 4000$ ran at two distinct speeds.
- **How far from optimal are you?** 1.5 FLOP per cycle against a peak of 32. OpenBLAS reaches 25. Then walk the ladder: remove the dependency chain, SIMD with FMA, register blocking, packing.
- **What would you change in the thread pool?** Its single queue gives dynamic load balancing, which made it far more robust than OpenBLAS's static split under interference (2% vs 33% variation at 16 threads). The cost is a mutex per task, which

shows at tiny tiles (–31% at tile 8). Fixes: batch tasks, or per-worker queues with work stealing.

- **Latency versus throughput?** An FMA takes 4 cycles but 2 can start every cycle, so you need 8 independent FMAs in flight. With 4 accumulators the microbenchmark ran at half speed.
- **How does this relate to GPUs?** Shared-memory tiling is cache blocking you manage by hand; warps hide latency the way SMT does, with far more threads. On both, register blocking gives the biggest gains: in Boehm's CUDA worklog, shared-memory tiling reached 12.8% of cuBLAS and register tiling 68.7%.

Claims to avoid

- "16 cores." It is 8 cores with 16 hardware threads.
- "Blocking made it 6× faster," without the size. It is 1.9× at $n = 2000$ and 6.2× at 4000.
- "Linear scaling." It is 80% efficiency at 8 workers.
- "It uses FMA." Today it compiles to a separate multiply and add, on purpose.
- "Hyperthreading always helps" or "never helps." It depends on what limits the kernel.
- Any number from one noisy run under WSL2, quoted without that context.

Resume lines

Keep the numbers honest and dated, and update them as the kernel improves:

- Built a C++17 thread pool and parallel cache-blocked matrix multiply; reached 15× over naive (58.7 GFLOP/s, 8-core Ryzen 7 9800X3D) and diagnosed scaling limits with hardware performance counters.
- Designed a benchmark suite with Google Benchmark: interleaved repetitions, median and CV reporting, $O(n^2)$ result verification, Mann-Whitney A/B tests and an OpenBLAS baseline.

12. Further reading

Start with three: MIT 6.172 lecture 1, CS:APP chapters 5 and 6, and Stanford CS149 lecture 2. Together they cover most of this guide. The rest is ordered roughly by how much it adds; links go to pages opened while writing this guide, and textbooks are named only.

Resource	Kind	Best for	Sections
MIT 6.172, Performance Engineering of Software Systems (Fall 2018)	Free course videos	Lecture 1: a matmul optimization ladder. 4: architecture. 10: measurement and timing. 14: cache-efficient algorithms.	1 to 6, 9
Bryant and O'Hallaron, <i>Computer Systems: A Programmer's Perspective</i> , 3rd ed.	Textbook	5.7 to 5.9: latency, throughput, accumulators. 6.2 to 6.6: locality, caches, the memory mountain. 9.6: TLB. 12.6: parallel speedup.	2, 3, 5
Stanford CS149, Parallel Computing (Fall 2024)	Free course slides	Lecture 2: SIMD and hardware multithreading on CPUs and GPUs in one framework. 5: scheduling. 7: GPU architecture and CUDA.	3 to 5, GPU sides
Hoefler and Belli, Scientific Benchmarking of Parallel Computing Systems (SC '15)	Paper	Twelve rules for measuring and reporting performance	6
Bakhvalov, Performance Analysis and Tuning on Modern CPUs	Free book	Hardware counters, profiling, top-down analysis	7
van de Geijn, Myers and Parikh, LAFF-On Programming for High Performance	Free course	A CPU matmul from loop orders to register blocking, packing and OpenMP	9
Goto and van de Geijn, Anatomy of High-Performance Matrix Multiplication (ACM TOMS 2008)	Paper	How BLAS libraries reach peak: packing and the micro-kernel	9
Boehm, How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance (2022)	Article	The same ladder on a GPU, measured step by step	9

Resource	Kind	Best for	Sections
Williams, Waterman and Patterson, Roofline (CACM 2009)	Paper	The roofline model and its ceilings	1
Drepper, What Every Programmer Should Know About Memory (2007)	Free long paper	Caches, TLBs and memory hardware in depth	2
Slotin, Algorithms for Modern Hardware	Free online book	Instruction-level parallelism, caches, SIMD, a matmul case study	3, 4, 9
Hennessy and Patterson, <i>Computer Architecture: A Quantitative Approach</i> , 6th ed.	Textbook	Ch. 2: memory hierarchy. Ch. 3: out-of-order execution. Ch. 4: SIMD, GPUs and the roofline side by side.	1 to 4
Hwu, Kirk and El Hajj, <i>Programming Massively Parallel Processors</i> , 4th ed.	Textbook	The CUDA side: memory hierarchy, tiling, occupancy, performance	GPU sides
Mytkowicz et al., Producing Wrong Data Without Doing Anything Obviously Wrong! (ASPLOS 2009)	Paper	Measurement bias from innocent setup changes	6
Georges, Buytaert and Eeckhout, Statistically Rigorous Java Performance Evaluation (OOPSLA 2007)	Paper	Statistics for performance comparisons	6
McSherry, Isard and Murray, Scalability! But at what COST? (HotOS 2015)	Paper	Why speedup needs a strong single-thread baseline	5
Herlihy and Shavit, <i>The Art of Multiprocessor Programming</i>	Textbook	Locks, concurrent queues, work stealing: the theory behind a better pool	5
Jain, <i>The Art of Computer Systems Performance Analysis</i>	Textbook	Experimental design and statistics for performance work	6

Resource	Kind	Best for	Sections
Gregg, <i>Systems Performance</i> , 2nd ed.	Textbook	Methodology and the <code>perf</code> tool	6, 7
Mitzenmacher and Upfal, <i>Probability and Computing</i>	Textbook	Section 1.3: verifying matrix multiplication (Freivalds)	6
Goldberg, <i>What Every Computer Scientist Should Know About Floating-Point Arithmetic</i> (1991)	Paper	Rounding and why reordering sums changes results	4
Google Benchmark user guide	Docs	The suite's framework: real time, repetitions, interleaving	6
Lemire on Zen 5 and Chips and Cheese on desktop Zen 5	Articles	Zen 5 details used here: the 448-entry reorder buffer, the 2-cycle add, full 512-bit FP units	3, 4